32

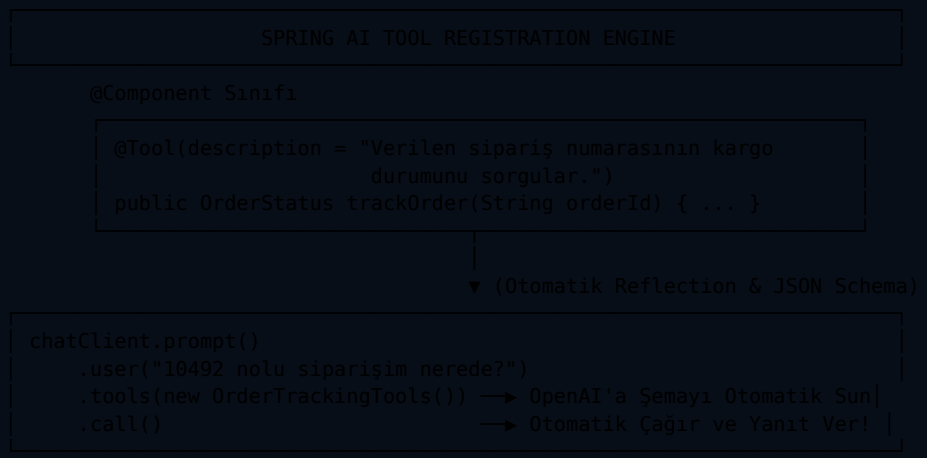
BÖLÜM: FUNCTION CALLING, TOOL USE VE API ENTEGRASYONU | MODULE: FUNCTION CALLING, TOOL INTEGRATION & AGENT TOOLING PROTOCOLS

Spring AI @Tool ve FunctionCallback Tabanlı Fonksiyon Kaydı

Spring AI Tool Calling: Declarative @Tool Annotations & FunctionCallback

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Evinize gelen bir tamirciye tek tek 'Tornavidanın adı şudur, pense şöyle çalışır' diye anlatmazsınız; alet çantasını önüne koyarsınız, o ihtiyacı olanı çantadan çekip kullanır. Spring AI'ın `\@Tool` mekanizması tam olarak budur! Siz sadece normal bir Java metodu yazıp üstüne `\@Tool(description = "Kargo takip et")` yazarsınız. Spring AI bu metodu otomatik inceler, açıklamasını OpenAI'ın anlayacağı dile çevirir ve model bu aracı istediği an otomatik çalıştırır!

🇬🇧 ENGLISH (ELI5)

When a certified technician arrives at your home, you don't explain how a wrench operates; you lay out the toolbox, and they grab the exact socket needed. Spring AI's `\@Tool` system turns ordinary Java methods into discoverable AI instruments! Simply annotate a method with `\@Tool(description = "Tracks package delivery by tracking ID")`. Spring AI uses reflection to synthesize the OpenAPI JSON schema and executes the method automatically when the LLM requests it!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Spring AI Tool Mekanizması: 1) `@Tool`: Metodun açıklamasını (`description`) taşır. LLM'in bu aracı ne zaman seçeceğini belirleyen EN KRİTİK unsur bu açıklamadır! 2) `@ToolParam`: Parametre açıklamalarını ve `required` durumunu belirler. 3) Otomatik Döngü (Auto-Resolution): Spring AI `ChatClient` varsayılan olarak tool çağrısını algılar, Java metodunu çalıştırır ve sonucu modele gönderip nihai metni kullanıcıya teslim eder; tüm döngüyü tek adımda çözer.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Spring AI Tool Calling Mechanics: 1) `@Tool` Annotation: Exposes method metadata to the LLM. The docstring `description` is the primary decision signal used by the model's router. 2) `@ToolParam`: Specifies parameter-level constraints, descriptions, and optionality. 3) Recursive Auto-Execution: Spring AI's execution pipeline intercepts the `tool_calls` finish reason, invokes the registered Java method via reflection, injects the result back into the prompt history, and re-invokes OpenAI seamlessly until text completion.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: @Tool ile Kargo Takip Servisi
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.ai.tool.annotation.ToolParam;
import org.springframework.stereotype.Service;

@Service
public class OrderTrackingTools {
    public record OrderStatus(String orderId, String status, String carrier, String estimatedDelivery)
{}

    @Tool(description = "Verilen sipariş numarasına göre kargo teslimat durumunu sorgular.")
    public OrderStatus getDeliveryStatus(
        @ToolParam(description = "8 haneli sipariş ID'si, örn: ORD-9481") String orderId
    ) {
        // Gerçek veritabanı veya kargo API çağrısı simülasyonu
        return new OrderStatus(orderId, "YOLDA", "Yurtiçi Kargo", "Yarın 14:00");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Kritik Açıklama Tuzağı: `@Tool` açıklamasını 'Sipariş fonksiyonu' gibi kısa ve belirsiz yazarsanız model bu aracı ne zaman çağıracağını anlayamaz ve kullanıcı sipariş sorduğunda aracı çağırmayıp halüsinasyon görür! Açıklama son derece detaylı ve örnekli yazılmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

The Ambiguous Tool Description Trap: If a tool's description is vague (e.g. 'Order helper'), the LLM's attention mechanism fails to trigger it, causing the model to hallucinate answers instead. Descriptions must explicitly state purpose, constraints, and valid invocation scenarios.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• \@Tool açıklaması, modelin doğru aracı seçmesini sağlayan en önemli rehberdir.

• Spring AI, Java metotlarını otomatik JSON Schema ve ToolManifest formatına çevirir.

• Auto-Resolution özelliği sayesinde fonksiyon çalıştırma döngüsü tek satırda tamamlanır.

🎯 Key Takeaways for the Interview

• The `\@Tool` description string is the primary routing signal for the model.

• Spring AI reflects Java method signatures into compliant OpenAPI tool schemas.

• Spring AI automates the execution-synthesis loop without manual plumbing.